

Groomsta - 4-Week Engineering Execution Plan

Version 1.0 MVP Development | Phase 1 Complete Delivery

Executive Summary

Timeline: 4 Weeks (28 Days)
Team Structure: 3 Core Developers + Incoming Interns
Objective: Deliver fully functional Phase 1 MVP with Customer, Partner, and Admin flows
Tech Stack: React/Next.js, Node.js/Express, PostgreSQL, Redis, Razorpay, Firebase, AWS S3

Team Responsibilities Overview

Developer	Core Domain	Accountability
Dev 1	Auth, Payments, Security	OTP system, Razorpay integration, JWT, data encryption, API security, wallet system
Dev 2	Customer Experience	Full customer UI/UX, booking flow, service discovery, order tracking, frontend-backend integration
Dev 3	Partner & Admin Systems	Partner dashboard, admin panel, verification workflow, job matching, payout management

WEEK 1: Foundation & Core Infrastructure

Week 1 Objectives

- Establish project infrastructure and development environment
- Build authentication systems for all user types
- Set up database schema and core backend APIs
- Create basic UI shells for Customer, Partner, and Admin

Dev 1: Authentication & Security Foundation

Responsibilities

- Multi-role authentication system (Customer, Partner, Admin)

- OTP verification via SMS gateway integration
- JWT token management with refresh tokens
- API security middleware and rate limiting
- Database encryption for sensitive fields

Deliverables

1. Authentication Module

- Phone OTP login flow (SMS provider: Twilio/MSG91)
- Email + password optional login
- Role-based access control (RBAC) middleware
- Session management with Redis

2. Security Infrastructure

- CORS configuration
- Helmet.js security headers
- Rate limiting (express-rate-limit)
- Input sanitization middleware
- Password hashing (bcrypt)

3. API Gateway Setup

- Centralized error handling
- Request validation (Joi/Zod)
- Logging system (Winston/Morgan)
- Environment variable management

Dependencies

- Backend repository initialized
- SMS gateway account (Twilio/MSG91) credentials
- Redis instance configured
- PostgreSQL database provisioned

Expected Outputs

- `/auth/send-otp` endpoint (POST)
- `/auth/verify-otp` endpoint (POST)

- `/auth/login` endpoint (POST)
- `/auth/refresh-token` endpoint (POST)
- Auth middleware: `authenticateUser`, `authorizeRole`
- **Security checklist document** for code reviews

Acceptance Criteria

- ✓ OTP sent successfully within 5 seconds
 - ✓ Token expiry handled with auto-refresh
 - ✓ Failed login attempts rate-limited (5 attempts/15 min)
 - ✓ All sensitive data encrypted in DB
 - ✓ Postman collection with all auth endpoints
-

Dev 2: Customer Interface Foundation

Responsibilities

- Customer-facing web application setup
- Landing page and service discovery UI
- Guest browsing experience
- Initial booking flow wireframes
- Frontend routing and state management

Deliverables

1. Frontend Project Setup

- Next.js project with TypeScript
- Tailwind CSS configuration
- Component library structure (Atomic Design)
- React Context/Redux for state management

2. Customer UI Pages (Static)

- Landing page (Home Services / Salon Services)
- Service categories listing page
- Service detail page (template)
- Login/OTP modal component
- Address input page (pincode + manual entry)

3. Backend APIs for Services

- GET `/api/services/categories`
- GET `/api/services/:categoryId/subcategories`
- GET `/api/services/:subcategoryId/services`
- GET `/api/services/:serviceId` (detail page)
- GET `/api/locations/check-serviceability?pincode=`

4. Integration Work

- Connect frontend to Dev 1's auth APIs
- API client setup (Axios/Fetch)
- Error boundary components

Dependencies

- Design assets (logos, icons, color palette)
- Service data structure from Admin
- Auth APIs from Dev 1

Expected Outputs

- Responsive landing page (mobile + desktop)
- Service browsing flow (3 levels deep)
- Guest user can browse without login
- OTP modal triggers on "Book Now"
- **Component documentation** (Storybook optional)

Acceptance Criteria

- ✓ Landing page loads in <2 seconds
- ✓ Service categories dynamically fetched from backend
- ✓ Pincode check returns serviceable status
- ✓ Mobile-responsive design (tested on 3 devices)
- ✓ Authentication flow integrated end-to-end

Dev 3: Partner & Admin Foundation

Responsibilities

- Partner onboarding flow (UI + backend)

- Admin dashboard skeleton
- Partner verification system
- Database schema for partners and bookings

Deliverables

1. Database Schema Implementation

- Users table (customers + partners + admin)
- Partners table (KYC details, bank info)
- Partner_Documents table (ID proof, photos)
- Services & Partner_Services tables
- Bookings table (initial structure)
- Service_Categories, Service_Subcategories

2. Partner Onboarding Backend

- POST `/api/partners/register`
- POST `/api/partners/upload-documents` (S3/Supabase)
- POST `/api/partners/bank-details`
- GET `/api/partners/profile`
- PUT `/api/partners/services` (add services offered)

3. Partner UI Pages

- Partner registration form (multi-step)
- Document upload component (drag-drop)
- Profile setup page
- Service selection & pricing page

4. Admin Dashboard (Basic)

- Admin login page
- Pending partner verifications list
- Partner detail view page (read-only)
- Approve/Reject actions (buttons only)

Dependencies

- Auth system from Dev 1

- S3 bucket or Supabase Storage setup
- Service catalog data

Expected Outputs

- Complete partner registration flow (7 steps)
- KYC documents uploaded to cloud storage
- Admin can view pending partners
- **Database migration scripts** (versioned)
- **ER diagram** updated with all relationships

Acceptance Criteria

- ✓ Partner can complete registration in <5 minutes
- ✓ Documents uploaded successfully (max 5MB each)
- ✓ Admin sees real-time pending applications
- ✓ Database handles 1000+ partner records efficiently
- ✓ Data validation on all input fields

Week 1: Cross-Team Deliverables

Deliverable	Owner	Status Checkpoint
Database fully migrated	Dev 3	Day 3
Auth APIs live on staging	Dev 1	Day 4
Landing page deployed	Dev 2	Day 5
Partner registration functional	Dev 3	Day 6
Week 1 Demo	All	Day 7 (Friday)

WEEK 2: Core Booking & Partner Matching System

Week 2 Objectives

- Complete customer booking flow (date/time selection)
- Build partner job request broadcast system
- Implement 20-second acceptance mechanism
- Develop order tracking infrastructure

- Payment gateway integration (Razorpay)
-

Dev 1: Payment Integration & Wallet System

Responsibilities

- Razorpay payment gateway integration
- Wallet system implementation
- Transaction logging and reconciliation
- Refund processing logic
- Payment security (PCI compliance basics)

Deliverables

1. Razorpay Integration

- POST `/api/payments/create-order` (Razorpay order)
- POST `/api/payments/verify` (signature verification)
- POST `/api/payments/webhook` (payment status updates)
- Payment success/failure callbacks

2. Wallet System

- Wallet table (balance, transactions)
- POST `/api/wallet/add-credits` (referral rewards)
- GET `/api/wallet/balance`
- POST `/api/wallet/deduct` (apply credits at checkout)
- Wallet transaction history

3. Payment Options Backend

- Full online payment
- Partial payment (advance + post-service)
- Pay after service (booking hold)

4. Refund System

- POST `/api/payments/refund`
- Razorpay refund API integration
- Refund to wallet or original source

Dependencies

- Razorpay account (Test + Live keys)
- Booking system from Dev 2
- SSL certificate for webhook endpoint

Expected Outputs

- Payment flow tested with ₹1 test transactions
- Wallet credits applied successfully
- Refund processed within 5-7 business days
- **Payment flow diagram** (Mermaid/Lucidchart)
- **Razorpay webhook handling guide**

Acceptance Criteria

- ✓ Payment success rate >99% (on staging)
 - ✓ Webhook retries handled (3 attempts)
 - ✓ Wallet balance updates in real-time
 - ✓ All payment data encrypted in transit and at rest
 - ✓ Transaction logs auditable
-

Dev 2: Complete Booking Flow & Tracking

Responsibilities

- Date and time selection UI
- Booking confirmation flow
- Payment integration (frontend)
- Order tracking page with real-time updates
- Customer dashboard (My Bookings)

Deliverables

1. Booking Flow Completion

- Date picker component (available dates only)
- Time window selection (Morning/Afternoon/Evening)
- Booking summary page (review before payment)
- Address confirmation step

- Add-ons selection UI

2. Payment Frontend

- Razorpay checkout integration
- Payment method selection page
- Payment success/failure screens
- Receipt download (PDF generation)

3. Customer Booking APIs

- POST `/api/bookings/create`
- GET `/api/bookings/:bookingId`
- GET `/api/bookings/user/:userId` (booking history)
- PUT `/api/bookings/:bookingId/cancel`

4. Order Tracking Page

- Status indicator (Accepted → On the Way → Started → Completed)
- Partner details card (name, photo, contact)
- Real-time status updates (polling or WebSocket)
- Contact partner button (call/message)

5. Customer Dashboard

- My Bookings tab (Active, Past, Cancelled)
- Booking detail view
- Rebook functionality
- Profile settings page

Dependencies

- Payment APIs from Dev 1
- Partner matching system from Dev 3
- Notification system (basic SMS/email)

Expected Outputs

- End-to-end booking flow (8 steps)
- Customer receives booking confirmation email
- Tracking page updates every 30 seconds

- **User journey testing checklist**
- **Mobile responsiveness report**

Acceptance Criteria

- ✓ Booking completed in <3 minutes
 - ✓ Payment gateway opens in <2 seconds
 - ✓ Order tracking shows real-time updates
 - ✓ Customer can cancel with appropriate refund
 - ✓ All booking data persisted correctly
-

Dev 3: Partner Job Matching & Admin Verification

Responsibilities

- Job broadcast system (20-second window)
- Partner acceptance/rejection logic
- Admin partner verification workflow
- Partner dashboard (incoming jobs)
- Real-time notification system

Deliverables

1. Job Broadcasting System

- Algorithm to find nearby partners (radius-based)
- POST `/api/bookings/broadcast` (internal trigger)
- WebSocket/Firebase for real-time job push
- 20-second countdown timer (frontend + backend)
- First-accept-wins logic

2. Partner Job APIs

- GET `/api/partners/jobs/incoming` (SSE or polling)
- POST `/api/partners/jobs/:jobId/accept`
- POST `/api/partners/jobs/:jobId/reject`
- GET `/api/partners/jobs/active`
- PUT `/api/partners/jobs/:jobId/status` (On the Way, Started, Completed)

3. Partner Dashboard UI

- Incoming job request card (real-time)
- Accept/Reject buttons with countdown
- Active jobs list
- Job detail view (customer address, services, payment)
- Status update controls

4. Admin Verification System

- Pending partners queue page
- Partner detail modal (view all KYC docs)
- Approve/Reject/Request More Info actions
- POST `/api/admin/partners/:partnerId/verify`
- Email notification on approval/rejection

5. Service Management (Admin)

- Service CRUD operations
- POST `/api/admin/services/create`
- PUT `/api/admin/services/:serviceId/update`
- Category and subcategory management

Dependencies

- Booking creation from Dev 2
- Notification system (email/SMS triggers)
- Geolocation library (Turf.js or PostGIS)

Expected Outputs

- Job broadcast to 10 partners in <3 seconds
- First partner acceptance locks the job
- Admin can verify 20 partners/hour
- **Partner matching algorithm documentation**
- **Admin user guide (PDF)**

Acceptance Criteria

- ✓ Job offered to partners within 5km radius
- ✓ 20-second timer accurate (± 1 second)

- ✔ No duplicate job assignments
- ✔ Admin approval triggers instant partner activation
- ✔ Partner notification latency <2 seconds

Week 2: Cross-Team Integration Points

Integration	Teams	Sync Meeting
Booking → Payment flow	Dev 2 ↔ Dev 1	Day 9
Booking → Job broadcast	Dev 2 ↔ Dev 3	Day 10
Partner acceptance → Customer notification	Dev 3 ↔ Dev 2	Day 11
Mid-sprint review	All	Day 14 (Friday)

WEEK 3: Advanced Features & Partner Management

Week 3 Objectives

- Partner service & pricing management
- Referral system implementation
- Membership plans
- Payout calculation logic
- Notification system (multi-channel)
- Customer reviews & ratings

Dev 1: Payout System & Financial Operations

Responsibilities

- Weekly payout calculation engine
- Commission deduction logic
- Bank transfer integration (Razorpay Payouts)
- Financial reports and reconciliation
- Referral credit distribution

Deliverables

1. Payout Calculation System

- Payout table (weekly cycles)
- Cron job for weekly payout generation
- POST `/api/payouts/calculate` (run every Sunday)
- GET `/api/payouts/partner/:partnerId`
- Commission calculation (% based on service type)

2. Razorpay Payout Integration

- POST `/api/payouts/initiate` (batch transfers)
- POST `/api/payouts/verify` (status check)
- Webhook for payout status updates
- Retry logic for failed transfers

3. Referral System Backend

- Referral_Codes table
- POST `/api/referrals/generate` (unique code per user)
- POST `/api/referrals/apply` (at signup/checkout)
- Referral reward trigger (₹100 to both)
- GET `/api/referrals/user/:userId/stats`

4. Membership System

- Membership_Plans table
- POST `/api/memberships/subscribe`
- Monthly subscription payment (Razorpay recurring)
- Discount application at checkout
- GET `/api/memberships/benefits`

Dependencies

- Razorpay Payout account activation
- Completed bookings data from Dev 2
- Partner bank details from Dev 3

Expected Outputs

- Payout report generated automatically

- Test payout of ₹10 to partner account
- Referral credits applied successfully
- **Financial reconciliation report template**
- **Payout SOP document**

Acceptance Criteria

- ✓ Payouts processed within 24 hours
 - ✓ Commission accuracy 100% (tested with 50 bookings)
 - ✓ Referral rewards credited instantly
 - ✓ Membership discounts auto-applied
 - ✓ All financial data auditable with timestamps
-

Dev 2: Customer Engagement & Reviews

Responsibilities

- Referral UI and sharing functionality
- Membership subscription flow
- Rating and review system
- Notifications integration (customer side)
- Profile and booking history enhancements

Deliverables

1. Referral System UI

- Referral section in customer dashboard
- Unique referral code display
- Share buttons (WhatsApp, SMS, Copy Link)
- Referral tracking (pending, successful)
- Notifications on referral success

2. Membership UI

- Membership plans page
- Subscription checkout flow
- Active membership indicator
- Benefits display (priority support, discounts)

3. Ratings & Reviews

- Post-service review prompt modal
- Star rating component (1-5 stars)
- Review text input (optional)
- POST `/api/reviews/create`
- GET `/api/reviews/service/:serviceId`
- Review display on service detail page

4. Notification Center

- In-app notification panel (bell icon)
- Notification list (read/unread)
- GET `/api/notifications/user/:userId`
- Mark as read functionality
- Notification preferences page

5. Customer Profile Enhancements

- Saved addresses management (CRUD)
- Wallet balance display
- Edit profile details
- Notification preferences

Dependencies

- Referral backend from Dev 1
- Review APIs
- Notification service (Firebase/OneSignal)

Expected Outputs

- Referral link shareable on WhatsApp
- Customer can subscribe to membership
- Reviews visible on service pages
- **A/B testing plan for referral incentives**
- **User engagement metrics dashboard**

Acceptance Criteria

- ✓ Referral link generates unique code
 - ✓ Membership payment successful
 - ✓ Reviews appear within 1 minute of submission
 - ✓ Notifications delivered in real-time
 - ✓ Profile updates saved successfully
-

Dev 3: Partner Operations & Admin Control

Responsibilities

- Partner service pricing management
- Partner availability settings
- Partner earnings dashboard
- Admin booking management
- Admin payout processing interface

Deliverables

1. Partner Pricing Management

- Service pricing page (view/edit)
- Price slider within allowed range ($\pm 20\%$)
- Add-ons management (enable/disable)
- PUT `/api/partners/services/:serviceId/pricing`
- Price comparison with other partners (optional)

2. Partner Availability System

- Working hours settings (day-wise)
- Availability toggle (online/offline)
- Block dates functionality (vacations)
- PUT `/api/partners/availability`
- GET `/api/partners/availability/:partnerId`

3. Partner Earnings Dashboard

- Current week earnings display
- Completed jobs count
- Commission breakdown
- Payout history table

- Earnings chart (weekly trend)

4. Admin Booking Management

- All bookings table (filters: status, date, partner)
- Booking detail view modal
- Cancel booking (admin override)
- Refund processing UI
- Disputed bookings section

5. Admin Payout Interface

- Weekly payout report generation
- Review payout amounts before release
- Initiate batch payout button
- Payout status tracking table
- Export payout report (CSV/PDF)

6. Admin Service Configuration

- Add/edit categories & subcategories
- Set base pricing for services
- Define partner pricing range
- Service visibility toggle (active/inactive)

Dependencies

- Payout calculation from Dev 1
- Booking data from Dev 2
- Partner service data

Expected Outputs

- Partner can adjust 10 service prices in <2 minutes
- Admin processes 50 payouts in <5 minutes
- Availability settings reflect in job matching
- **Partner earnings calculation formula doc**
- **Admin operations manual**

Acceptance Criteria

- ✔ Partner price updates reflected immediately
- ✔ Availability toggle affects job assignments
- ✔ Admin can filter 1000+ bookings efficiently
- ✔ Payout reports match ledger 100%
- ✔ Service configuration saves without errors

Week 3: Quality Assurance Checkpoints

QA Task	Owner	Deadline
API response time audit (<500ms)	Dev 1	Day 18
Frontend performance (Lighthouse >90)	Dev 2	Day 19
Partner dashboard load testing (100 concurrent)	Dev 3	Day 19
Cross-browser testing (Chrome, Safari, Firefox)	Dev 2	Day 20
UAT with 5 test users	All	Day 21 (Friday)

WEEK 4: Notifications, Testing & Deployment

Week 4 Objectives

- Multi-channel notification system (SMS, WhatsApp, Email, Push)
- Comprehensive testing (unit, integration, E2E)
- Bug fixes and performance optimization
- Production deployment preparation
- Documentation and handover

Dev 1: Notification Engine & Final Security Audit

Responsibilities

- Unified notification service (all channels)
- Webhook integrations (WhatsApp, SMS)
- Email templates and delivery
- Security audit and penetration testing
- Production environment setup

Deliverables

1. Notification Service

- Notification queue system (Bull/BullMQ)
- SMS integration (Twilio/MSG91)
- WhatsApp Business API integration (Gupshup/Twilio)
- Email service (SendGrid/AWS SES)
- Push notifications (Firebase Cloud Messaging)

2. Notification Templates

- OTP SMS template
- Booking confirmation (WhatsApp + Email)
- Partner assigned notification
- Payment receipt email
- Payout processed SMS

3. Notification APIs

- POST `/api/notifications/send` (internal)
- POST `/api/notifications/schedule` (delayed send)
- GET `/api/notifications/logs`
- Retry logic for failed notifications

4. Security Audit

- SQL injection testing (all endpoints)
- XSS vulnerability scan
- CSRF token implementation
- API rate limiting verification
- Data encryption audit (at rest + in transit)
- **Security audit report**

5. Production Setup

- Environment variables (.env.production)
- Database backup strategy (daily automated)
- SSL certificate installation
- CDN configuration (Cloudflare)

- Error tracking (Sentry/Rollbar)

Dependencies

- WhatsApp Business API approval
- Email domain verification (SPF, DKIM)
- Production server access (AWS/DigitalOcean)

Expected Outputs

- 100 test notifications sent successfully
- All endpoints pass security scan
- Production database migrated
- **Incident response playbook**
- **API documentation (Swagger/Postman)**

Acceptance Criteria

- ✓ SMS delivered within 10 seconds
 - ✓ WhatsApp messages 95% delivery rate
 - ✓ Email deliverability >98%
 - ✓ No critical security vulnerabilities
 - ✓ Production environment stable (99.9% uptime target)
-

Dev 2: Frontend Testing & Optimization

Responsibilities

- End-to-end testing (Cypress/Playwright)
- Performance optimization
- Mobile PWA features
- User acceptance testing coordination
- Frontend deployment

Deliverables

1. E2E Testing Suite

- Test: Guest booking flow (10 test cases)
- Test: Registered user booking (5 test cases)

- Test: Payment success/failure scenarios
- Test: Order tracking updates
- Test: Referral and membership flows

2. Performance Optimization

- Image lazy loading and compression
- Code splitting (route-based)
- Bundle size reduction (<500KB initial load)
- Lighthouse audit (score >90)
- Caching strategy (service workers)

3. PWA Features

- Add to home screen prompt
- Offline fallback page
- Push notification permission request
- Web app manifest configuration
- Splash screen and app icons

4. Responsive Testing

- Test on 5 device sizes
- iOS Safari compatibility
- Android Chrome compatibility
- Tablet view optimization

5. User Acceptance Testing

- Coordinate with 10 beta testers
- Collect feedback forms
- Bug reporting template
- Prioritize critical bugs

Dependencies

- Backend APIs stable (Dev 1, Dev 3)
- Test accounts created
- Staging environment live

Expected Outputs

- 50+ E2E test cases passing
- Frontend loads in <2 seconds (3G network)
- PWA installable on mobile
- **Bug tracking sheet (Jira/Linear)**
- **UAT feedback summary report**

Acceptance Criteria

- ✓ All E2E tests pass on staging
 - ✓ Lighthouse performance score >90
 - ✓ PWA passes Chrome Audit
 - ✓ Zero critical bugs in UAT
 - ✓ Mobile experience rated 4.5/5 by testers
-

Dev 3: Partner & Admin Testing + Deployment

Responsibilities

- Partner dashboard testing
- Admin panel final features
- Job matching algorithm optimization
- Database performance tuning
- Backend deployment and monitoring

Deliverables

1. Partner Testing

- Test: Partner registration to approval (5 partners)
- Test: Job acceptance race condition (10 partners, 1 job)
- Test: Status updates sync with customer
- Test: Earnings calculation accuracy
- Load test: 100 partners online simultaneously

2. Admin Final Features

- Analytics dashboard (bookings, revenue, partners)
- Export reports (CSV for bookings, payouts, partners)

- Coupon management system (CRUD)
- Membership plan editor
- System logs viewer

3. Job Matching Optimization

- Indexing on location fields (PostGIS)
- Caching eligible partners (Redis)
- Reduce broadcast latency to <2 seconds
- Handle edge cases (no partners available)

4. Database Optimization

- Query performance analysis
- Add indexes on frequently queried fields
- Partitioning for bookings table (by date)
- Connection pooling configuration
- **Database optimization report**

5. Backend Deployment

- Docker containerization
- CI/CD pipeline (GitHub Actions)
- Auto-scaling configuration (Kubernetes/PM2)
- Health check endpoints
- Monitoring dashboard (Grafana/Datadog)

Dependencies

- Load testing tools (K6/JMeter)
- Production database credentials
- Server access for deployment

Expected Outputs

- Partner dashboard handles 500 concurrent users
- Admin can generate reports in <5 seconds
- Job matching <3 seconds end-to-end
- **System architecture diagram (updated)**

- **Deployment runbook**

Acceptance Criteria

- ✓ Partner acceptance rate >80% in tests
 - ✓ Admin dashboard loads all data <3 seconds
 - ✓ Job matching works with 1000+ partners
 - ✓ Database queries optimized (<100ms avg)
 - ✓ Backend deployed with zero downtime
-

Week 4: Final Integration & Go-Live

Day 22-23: Integration Testing

- All three systems tested together
- End-to-end scenarios (10 critical paths)
- Load testing with realistic traffic simulation
- Fix integration bugs

Day 24-25: Bug Bash & Fixes

- Entire team tests all features
- Prioritize P0 (critical) bugs
- Regression testing after fixes
- Performance bottleneck resolution

Day 26: Pre-Production Deployment

- Deploy to staging environment
- Final smoke testing
- Database migration dry-run
- Rollback plan documented

Day 27: Production Deployment

- Deploy backend (Dev 1, Dev 3)
- Deploy frontend (Dev 2)
- Monitor logs and metrics
- On-call rotation established

Day 28: Go-Live & Monitoring

- Announce launch internally
 - Monitor first 24 hours closely
 - Incident response team active
 - **Post-mortem if issues arise**
-

Final Deliverables Checklist

Technical Documentation

- ☐ API documentation (Swagger/Postman)
- ☐ Database schema and ER diagram
- ☐ System architecture diagram
- ☐ Deployment runbook
- ☐ Security audit report
- ☐ Environment setup guide

User Documentation

- ☐ Customer user guide
- ☐ Partner onboarding guide
- ☐ Admin operations manual
- ☐ FAQ document

Code & Repository

- ☐ All code merged to main branch
- ☐ Code review completed (100% coverage)
- ☐ Unit test coverage >70%
- ☐ E2E test suite executable
- ☐ Git tags for v1.0 release

Operations

- ☐ Monitoring dashboards configured
 - ☐ Alert rules set up (PagerDuty/Slack)
 - ☐ Backup and restore tested
 - ☐ Incident response plan
 - ☐ On-call schedule (Week 5+)
-

Success Metrics (Week 4 End)

Metric	Target	Measurement
Customer Bookings	10 test bookings completed	Real transactions on staging
Partner Acceptance	>80% acceptance rate	20-second window tests
Payment Success	>99% success rate	Razorpay test transactions
Page Load Time	<2 seconds	Lighthouse audit
API Response Time	<500ms (p95)	Load testing results
Uptime	99.9%	First 7 days post-launch
Bug Count	<5 P0 bugs	Bug tracking system

Risk Mitigation Plan

High-Risk Areas

1. 20-Second Job Matching

- Mitigation: Pre-cache eligible partners, use Redis pub/sub
- Fallback: Extend window to 30 seconds if needed

2. Payment Gateway Downtime

- Mitigation: Razorpay has 99.95% uptime SLA
- Fallback: Queue payments, retry automatically

3. Partner No-Show

- Mitigation: Automated re-broadcast system
- Fallback: Customer service manual assignment

4. Database Performance

- Mitigation: Indexing, query optimization, read replicas
- Fallback: Vertical scaling (upgrade instance)

Communication Plan

- Daily Standups:** 10 AM (15 minutes)
- Weekly Demo:** Friday 4 PM (1 hour)
- Slack Channel:** #groomsta-dev (real-time issues)
- Blocker Resolution:** <4 hours turnaround

Post-Week 4: Maintenance Mode

Week 5 Priorities

1. Monitor production metrics
2. User feedback incorporation
3. Minor bug fixes
4. Performance tuning
5. Plan Phase 2 features

Onboarding New Interns

- Assign to specific modules (referrals, reviews, PWA)
- Pair programming with core devs
- Code review training
- Take over low-priority features

Appendix: Technology Stack Confirmation

Layer	Technology	Purpose
Frontend	Next.js 14 (React 18)	SSR, SEO, Performance
Styling	Tailwind CSS	Rapid UI development
State	Redux Toolkit / React Context	Global state management
Backend	Node.js 20 + Express.js	API server
Database	PostgreSQL 15	Primary datastore
Cache	Redis 7	Session, real-time data
Storage	AWS S3 / Supabase Storage	Documents, images
Payments	Razorpay	Payment gateway
Auth	JWT + Twilio/MSG91	OTP verification
Notifications	Firebase FCM	